UNITY 6000.0.62F1 · COMPLETE OFFLINE DOCUMENTATION

Jutsu Defender Developer Guide

Install. Edit. Reskin. Build your release.

szekipapa77@gmail.com



CHAPTER 01

Start here: Jutsu Defender

Jutsu Defender is a standalone editable Unity game template. Tap hand signs or press Q W E R to form three-sign jutsu combinations. Q W E casts Fire Orb. Drag to look around, or use A and D in the desktop layout. Stop bandits before they reach you and avoid hitting villagers.

Includes 25 curated levels, source-backed generation, a game-specific visual editor and validation, 3 guided helpers, local saving and portrait/landscape scenes. Keep the required private game Kit and publisher-owned Szekipapa77Ads layer. Provider SDK binaries are excluded. There is no bundled HLK, purchases, multiplayer server or remote LiveOps backend. Import only one game Kit into a project because the Kit classes share names.

CHAPTER 02

Requirements, installation and first play

Use Unity **6000.0.62f1**, URP **17.0.4** and Unity UI **2.0.0**. Install matching Android SDK/NDK/OpenJDK and WebGL modules in Unity Hub. Retain the supplied manifest and ProjectSettings.

1. Extract the standalone project to a short writable local path, then open it in the matching editor. Wait for package import and compilation.
2. For native unitypackage import, create a matching URP project and use Assets > Import Package > Custom Package. Import every game, Kit, Szekipapa77Ads and WebGL-template entry.
3. Run Szekipapa77 > Jutsu Defender > Setup. It configures rendering and player settings; review its changes when integrating with existing work.
4. Active Input Handling must support the classic Input Manager: choose Old or Both. New Input System only is insufficient.
5. Open `Assets/Games/ShadowHands/Scenes/ShadowHands_Mobile.unity` and press Play. Follow the level card and first-level instructions.
6. Stop Play mode before editing level assets. Keep all .meta GUIDs. The game uses classic Unity UI Text; do not assume an extra TextMesh Pro import is required.

CHAPTER 03

Project folders and exact source map

All game-relative paths below start at `Assets/Games/ShadowHands`. Runtime UI is built in code; editing an unrelated static Canvas does not reskin it. Scripts contains game rules and views; Editor contains authoring tools; Kit supplies UI/audio/save/platform support; Resources/shadowhands contains level JSON, art, music and sound; Settings owns URP assets; Scenes contains both entry scenes.

- `Scripts/ShadowHandsGame.cs`
- `Scripts/ShadowHandsGame.Hands.cs`
- `Scripts/ShadowHandsGame.UI.cs`
- `Scripts/ShadowHandsGame.View.cs`
- `Scripts/ShBot.cs`
- `Scripts/ShData.cs`
- `Scripts/ShGen.cs`
- `Scripts/ShLevels.cs`
- `Scripts/ShMeshes.cs`
- `Scripts/ShSfx.cs`
- `Scripts/ShSim.cs`

Main flow: ShadowHandsGame.cs. UI and popup consumers: ShadowHandsGame.UI.cs. Model/generator/loading: ShData.cs, ShGen.cs, ShLevels.cs, ShSim.cs. Authoring: Editor/ShadowHandsLevelEditor.cs. Shared ad settings live outside the game at Assets/Szekipapa77Ads/Resources/AdsSettings.asset.

CHAPTER 04

Gameplay, controls and progression

Tap hand signs or press Q W E R to form three-sign jutsu combinations. Q W E casts Fire Orb. Drag to look around, or use A and D in the desktop layout. Stop bandits before they reach you and avoid hitting villagers.

The initial bank has 25 curated levels. Read the supplied loader/generator before changing continuation rules. Play the guided opening, ordinary progress, completion and retry paths; preserve saved indexes during a visual reskin. The Mobile scene is portrait and the WebGL-named scene is landscape, but the supplied Build menu selects portrait for both build targets. Select landscape explicitly in your Build Profile if desired. Test a near-startup scene replacement as well as a normal restart.

CHAPTER 05

Tutorials and every helper

Fresh local progress triggers the first-level guidance. Newly unlocked helpers have introduction and guided-use flows. Reset only a test save when reviewing tutorials. A free guided first use can differ from ordinary charge consumption. Exact helper names/unlocks/charges: Scripts/ShData.cs; help and effect descriptions: Scripts/ShData.cs.

Helper	Unlock	Free charges	Action and effect
Spirit Wolf	3	2	A spirit wolf bites the three nearest enemies.
Frost Scroll	6	2	Freezes every enemy on this step for 4 seconds.
Meteor Rain	11	2	Meteors wipe out every bandit on this step and badly hurt a Brute.

Activate the tool and perform its required target action: arming a tool alone does not demonstrate its effect. A successful use requires the stated preconditions from this game: show both activation and its actual effect. Exhausted charges may offer a rewarded ad; verify success/cancel/failure and grant a reward once only.

CHAPTER 06

Visual level editor: worked editing session

Open **Szekipapa77 > Jutsu Defender > Level Editor**. Exact implementation: `Editor/ShadowHandsLevelEditor.cs`.

Duplicate an early entry in this game-specific visual editor, change its visible name and one rule, save, validate, then play it. Use the exact source-control table below for this game.

1. Back up Resources/shadowhands/Levels/levels.json.
2. Duplicate a simple early level and edit one small element.
3. Use the game-specific validation controls and resolve every reported issue.
4. Save explicitly; stop and restart Play mode, then use Play this level or navigate to the edited index.
5. Check portrait and landscape and rebuild demo outputs after accepting data changes.

The exact current authoring controls, including tooltips and generator switches, follow. These are game-specific source references, not a generic editor feature list.

Source	Exact current code
`ShadowHandsLevelEditor.cs:44`	`[MenuItem("Szekipapa77/Jutsu Defender/Level Editor")]`
`ShadowHandsLevelEditor.cs:47`	`[MenuItem("Szekipapa77/Jutsu Defender/Validate All Levels")]`
`ShadowHandsLevelEditor.cs:120`	`if (GUILayout.Button(_dirty ? "Save*" : "Save", EditorStyles.toolbarButton, GUILayout.Width(70))) Save();`
`ShadowHandsLevelEditor.cs:121`	`if (GUILayout.Button("Reload", EditorStyles.toolbarButton, GUILayout.Width(60)))`
`ShadowHandsLevelEditor.cs:126`	`if (GUILayout.Button("Validate", EditorStyles.toolbarButton, GUILayout.Width(70)) && Cur != null) ValidateOne(_sel);`
`ShadowHandsLevelEditor.cs:127`	`if (GUILayout.Button("Validate All", EditorStyles.toolbarButton, GUILayout.Width(85))) ValidateAll();`
`ShadowHandsLevelEditor.cs:131`	`if (GUILayout.Button("▶ Play this level", EditorStyles.toolbarButton, GUILayout.Width(120)) && Cur != null) Play(_sel + 1);`

ShadowHandsLevelEditor.cs:157	`if (GUILayout.Button("Add")) { var list = new List<LevelData>(_file.levels); list.Insert(_sel + 1 > list.Count ? list.Count : _sel + 1, NewLevel()); _file.levels = list.ToArray(); _sel = Mathf.Min(_sel + 1, _file.levels.Length - 1); _step = 0; _dirty = true; _results.Clear(); }`
ShadowHandsLevelEditor.cs:158	`if (GUILayout.Button("Dup") && Cur != null) { var list = new List<LevelData>(_file.levels); var c = Cur.Clone(); c.name += " copy"; list.Insert(_sel + 1, c); _file.levels = list.ToArray(); _sel++; _dirty = true; _results.Clear(); }`
ShadowHandsLevelEditor.cs:159	`if (GUILayout.Button("Del") && Cur != null && EditorUtility.DisplayDialog("Delete level", $"Delete level {_sel + 1} '{Cur.name}'?", "Delete", "Cancel"))`
ShadowHandsLevelEditor.cs:163	`if (GUILayout.Button("▲ Up") && _sel > 0) { Swap(_sel, _sel - 1); _sel--; }`
ShadowHandsLevelEditor.cs:164	`if (GUILayout.Button("▼ Down") && _sel < _file.levels.Length - 1) { Swap(_sel, _sel + 1); _sel++; }`
ShadowHandsLevelEditor.cs:183	`l.name = EditorGUILayout.TextField("Name", l.name);`
ShadowHandsLevelEditor.cs:184	`l.theme = EditorGUILayout.Popup("Theme", l.theme, ThemeNames);`
ShadowHandsLevelEditor.cs:185	`l.difficulty = EditorGUILayout.Slider("Design difficulty", l.difficulty, 0, 1);`
ShadowHandsLevelEditor.cs:186	`l.breather = EditorGUILayout.Toggle("Breather", l.breather);`
ShadowHandsLevelEditor.cs:187	`l.tut = EditorGUILayout.IntPopup("Tutorial", l.tut, new[] { "None", "Sign tutorial", "Look tutorial" }, new[] { 0, 1, 2 });`
ShadowHandsLevelEditor.cs:188	`l.hint = EditorGUILayout.TextField("First-time hint", l.hint);`
ShadowHandsLevelEditor.cs:192	`EditorGUILayout.HelpBox($"Bot: {v}", v.Solvable ? MessageType.Info : MessageType.Error);`

`ShadowHandsLevelEditor.cs:203`	`if (GUILayout.Button($"Step {s + 1}", style, GUILayout.Width(70))) { _step = s; _pickKind = -1; }`
`ShadowHandsLevelEditor.cs:206`	`if (GUILayout.Button("+ Step", GUILayout.Width(60)))`
`ShadowHandsLevelEditor.cs:211`	`if (l.steps.Length > 1 && GUILayout.Button("Dup step", GUILayout.Width(70)))`
`ShadowHandsLevelEditor.cs:215`	`if (l.steps.Length > 1 && GUILayout.Button("- Step", GUILayout.Width(60)))`
`ShadowHandsLevelEditor.cs:226`	`st.turn = EditorGUILayout.Slider(_step == 0 ? "Turn (first step: none)" : "Turn from previous (°)", _step == 0 ? 0 : st.turn, -90, 90);`
`ShadowHandsLevelEditor.cs:241`	`_newDelay = EditorGUILayout.Slider("New enemy delay (s)", _newDelay, 0, 12);`
`ShadowHandsLevelEditor.cs:242`	`EditorGUILayout.HelpBox("Top-down arena: you stand at the bottom centre looking up. Left-click empty ground to place the brush, drag to move, right-click to delete. Rings every 5 m.", MessageType.None);`
`ShadowHandsLevelEditor.cs:400`	`s.type = EditorGUILayout.Popup("Enemy", s.type, EnemyType.Name);`
`ShadowHandsLevelEditor.cs:401`	`s.a = EditorGUILayout.Slider("Angle (°)", s.a, -100, 100);`
`ShadowHandsLevelEditor.cs:402`	`s.d = EditorGUILayout.Slider("Distance (m)", s.d, 2.5f, ArenaRange);`
`ShadowHandsLevelEditor.cs:403`	`s.delay = EditorGUILayout.Slider("Delay (s)", s.delay, 0, 15);`
`ShadowHandsLevelEditor.cs:404`	`if (s.type == EnemyType.Brute) s.hp = EditorGUILayout.IntSlider("Brute HP", s.hp <= 0 ? EnemyType.DefaultHp(s.type) : s.hp, 3, 20);`
`ShadowHandsLevelEditor.cs:406`	`EditorGUILayout.LabelField("Reaches you after", $"{eta:0.0} s");`
`ShadowHandsLevelEditor.cs:413`	`p.type = EditorGUILayout.Popup("Prop", p.type, PropType.Name);`
`ShadowHandsLevelEditor.cs:414`	`p.a = EditorGUILayout.Slider("Angle (°)", p.a, -100, 100);`

ShadowHandsLevelEditor.cs:415	p.d = EditorGUILayout.Slider("Distance (m)", p.d, 2.5f, ArenaRange);
ShadowHandsLevelEditor.cs:436	_genCount = EditorGUILayout.IntField("Count", _genCount); _genCount = Mathf.Clamp(_genCount, 1, 50);
ShadowHandsLevelEditor.cs:437	_genStart = EditorGUILayout.IntField("Start level #", _genStart); _genStart = Mathf.Max(1, _genStart);
ShadowHandsLevelEditor.cs:440	_genSeed = EditorGUILayout.IntField("Seed", _genSeed);
ShadowHandsLevelEditor.cs:441	_genBreather = EditorGUILayout.IntSlider("Breather every", _genBreather, 0, 10);
ShadowHandsLevelEditor.cs:443	EditorGUILayout.MinMaxSlider($"Difficulty {_genFrom:0.00} → {_genTo:0.00}", ref _genFrom, ref _genTo, 0.05f, 1f);
ShadowHandsLevelEditor.cs:454	_genReplace = EditorGUILayout.ToggleLeft("Replace levels from the start level (otherwise append)", _genReplace);
ShadowHandsLevelEditor.cs:455	if (GUILayout.Button($"Generate {_genCount} bot-validated levels", GUILayout.Height(28))) Generate();

CHAPTER 07

Generator, insertion and replacement

Use this editor generation panel, choose a reproducible seed and small count, review start/index and replace behavior, then generate, save and reopen. The source excerpt below identifies exact append/replace semantics.

Source	Exact current code
`ShadowHandsLevelEditor.cs:42`	`bool _genReplace;`
`ShadowHandsLevelEditor.cs:454`	`_genReplace = EditorGUILayout.ToggleLeft("Replace levels from the start level (otherwise append)", _genReplace);`
`ShadowHandsLevelEditor.cs:474`	`int at = _genReplace ? Mathf.Clamp(_genStart - 1, 0, list.Count) : list.Count;`
`ShadowHandsLevelEditor.cs:475`	`if (_genReplace) list.RemoveRange(at, Mathf.Min(gen.Count, list.Count - at));`
`ShadowHandsLevelEditor.cs:476`	`list.InsertRange(at, gen);`

Worked extension: back up the bank, generate a small set at moderate difficulty using the exact per-game controls, validate them, then Save. Reopen the JSON and confirm the count changed. Do not assume every editor has the same insertion behavior: the actual implementation below distinguishes insert, append and replace. Runtime continuation does not automatically append new records to project JSON. Validation and bot scores help find problems but do not certify a pleasant human difficulty curve.

CHAPTER 08

Actual level JSON and identifiers

Canonical asset: `Assets/Games/ShadowHands/Resources/shadowhands/Levels/levels.json`. Array order controls visible numbering. Preserve numeric/boolean types, root version fields where present and resource/save identifiers. The actual first record is shown below; use the editor for structured coordinates and block/word data.


```
{
 "name": "First Sign",
 "theme": 0,
 "difficulty": 0.05,
 "breather": false,
 "tut": 1,
 "hint": "",
 "steps": [
   {
     "turn": 0,
     "spawns": [
       {
         "type": 0,
         "a": 0,
         "d": 11,
         "delay": 0,
         "hp": 0
       }
     ],
     "props": []
   },
   {
     "turn": 0,
     "spawns": [
       {
         "type": 0,
         "a": -8,
         "d": 12,
         "delay": 1.5,
         "hp": 0
       },
       {
         "type": 0,
         "a": 8,
         "d": 12,
         "delay": 3.5,
         "hp": 0
       }
     ],
     "props": []
   }
 ]
}
```


Root keys: levels. Inspect ShData.cs, ShGen.cs, ShLevels.cs, ShSim.cs for field semantics and validation. Keep visible names/hints distinct from technical keys. The game-specific board, wave, card, opponent or question records are gameplay data; UI translation is separate.

CHAPTER 09

Reskin recipe: interface colours

Stop Play mode and open `Scripts/ShadowHandsGame.UI.cs`. Find `SignHex`, currently `#3D7BFF`. Change that hex literal to `#42C9B0`, preserve its symbol/type, compile and reopen the scene. Review HUD, menus, helper cards, settings/results and both orientations. Art sprites and procedural fallback shapes can have additional colours; edit them deliberately.

Actual consumers of this symbol are listed first, then all current palette/hex assignments. Editor-only palettes do not control the runtime. Preserve state/goal contrast and rebuild after accepting a change.

Source	Exact current code
`ShadowHandsGame.UI.cs:11 / class/static declarations`	`static readonly string[] SignHex = { "#3D7BFF", "#35C76B", "#FFC531", "#A35BFF" };`
`ShadowHandsGame.UI.cs:202 / void BuildComboScroll(bool portrait)`	`var chip = HzUI.Panel("Chip", row.transform, HzUI.Hex(SignHex[sign]), 24);`
`ShadowHandsGame.UI.cs:236 / void BuildSigns(bool portrait)`	`var glow = HzUI.Panel("Glow", _hud, HzUI.Hex(SignHex[k]), (int)(size * 0.6f));`
`ShadowHandsGame.UI.cs:241 / void BuildSigns(bool portrait)`	`var b = HzUI.Button("Sign" + k, _hud, null, locked ? new Color(0.45f, 0.45f, 0.5f) : HzUI.Hex(SignHex[k]), Color.white, () => PressSign(kk), 40, (int)(size * 0.5f));`
`ShadowHandsGame.UI.cs:339 / void UpdateHud()`	`var col = HzUI.Hex(SignHex[Skill.Combo[k][c]]);`
`ShadowHandsGame.UI.cs:629 / void ShowSkills()`	`var chip = HzUI.Panel("Chip", card.transform, HzUI.Hex(SignHex[sign]), 26);`

Source	Exact current code
`ShadowHandsGame.cs:326 / public void UseBooster(int b)`	`if (ok) { SetCharges(b, Charges(b) + 1); HzAudio.Play(Sfx.Coin, 0.7f); Toast($"+1 {Booster.Name[b]}", HzUI.Hex("#FFE27A"), 1.2f, 0.62f); RefreshHud(); }`
`ShadowHandsGame.cs:353 / IEnumerator WinFlow()`	`Banner("CLEAR!", HzUI.Hex("#3FE08A"));`
`ShadowHandsGame.cs:367 / IEnumerator LoseFlow()`	`Banner("DEFEATED!", HzUI.Hex("#FF5A4E"));`
`ShadowHandsGame.cs:391 / public bool TryUpgrade(int k)`	`public static readonly string[] GloveHex = { "#2B2F3A", "#B3263A", "#1E8C66", "#C99A2E", "#6A3FC2" };`

ShadowHandsGame.cs:392 / public bool TryUpgrade(int k)	public static readonly string[] BandHex = { "#19C3A6", "#FFD34D", "#FFFFFF", "#2B2F3A", "#FF7AD9" };
ShadowHandsGame.Hands.cs:54 / void BuildHands()	_gloveMat = new Material(Lit("#2B2F3A", 0.35f));
ShadowHandsGame.Hands.cs:55 / void BuildHands()	_bandMat = new Material(Lit("#19C3A6", 0.5f));
ShadowHandsGame.Hands.cs:56 / void BuildHands()	_skinMat = Lit("#F1C39B", 0.35f);
ShadowHandsGame.Hands.cs:58 / void BuildHands()	_chargeOrb = Obj("Charge", _handsRoot, ShMeshes.Sphere(14), Glow("#FF8A2A", 2.4f), new Vector3(0, -0.15f, 0.52f), Vector3.zero, null, false);
ShadowHandsGame.Hands.cs:117 / void OnFizzleView()	Puff(_cam.transform.TransformPoint(new Vector3(0, -0.15f, 0.55f)), "#9AA4AE", 0.35f, 0.4f);
ShadowHandsGame.Hands.cs:124 / void OnChargeView(int skill)	string hex = skill == Skill.Fire ? "#FF8A2A" : skill == Skill.Thunder ? "#B07CFF" : skill == Skill.Stars ? "#DDE6EE" : "#BFF6FF";
ShadowHandsGame.UI.cs:11 / class/static declarations	static readonly string[] SignHex = { "#3D7BFF", "#35C76B", "#FFC531", "#A35BFF" };
ShadowHandsGame.UI.cs:15 / class/static declarations	static readonly string[] BoostHex = { "#19C3A6", "#4FB7FF", "#FF6A3D" };
ShadowHandsGame.UI.cs:58 / void CoinPill(Transform parent, Vector2 anchor, Vector2 pos)	var coin = HzUI.Panel("Coin", bg.transform, HzUI.Hex("#FFC531"), 30);
ShadowHandsGame.UI.cs:60 / void CoinPill(Transform parent, Vector2 anchor, Vector2 pos)	coin.AddOutline(HzUI.Hex("#C98A0F"), 3);
ShadowHandsGame.UI.cs:100 / void BuildHome()	var play = Pill(_home, "Play", "#FFA51F", () => { if (_mode == Mode.Home) { Destroy(_home.gameObject); StartPlay(); } },
ShadowHandsGame.UI.cs:104 / void BuildHome()	var skills = Tab(_home, "SKILLS", HzIcons.Bolt, "#FF5A4E", ShowSkills, true, p);
ShadowHandsGame.UI.cs:107 / void BuildHome()	var dot = HzUI.Panel("New", skills.transform, HzUI.Hex("#FFE14D"), 22);
ShadowHandsGame.UI.cs:111 / void BuildHome()	Tab(_home, "GLOVES", HzIcons.TouchApp, "#8A4DFF", ShowGloves, false, p);

`ShadowHandsGame.UI.cs:283 / void BuildBoosters(bool portrait)`	`var badge = HzUI.Panel("Badge", btn.transform, HzUI.Hex("#FF4D4D"), 24);`
`ShadowHandsGame.UI.cs:314 / void RefreshHud()`	`badge.GetComponent<Image>().color = c > 0 ? HzUI.Hex("#FF4D4D") : HzUI.Hex("#35C76B");`
`ShadowHandsGame.UI.cs:364 / void RefreshStepDots()`	`_dots[i].color = done ? HzUI.Hex("#3FE08A") : cur ? HzUI.Hex("#FFC531") : new Color(1, 1, 1, 0.55f);`
`ShadowHandsGame.UI.cs:380 / void LevelBanner()`	`var s = HzUI.Label("S", bg.transform, _def.name, 46, HzUI.Hex("#FFE27A")); s.rectTransform.Place(new Vector2(0.5f, 0), new Vector2(800, 70), new Vector2(0, 40));`
`ShadowHandsGame.UI.cs:429 / void ChainPop(int n)`	`var t = HzUI.Label("Chain", Overlay, $"CHAIN x{n}", 90, HzUI.Hex("#FFE14D"));`
`ShadowHandsGame.UI.cs:445 / void FloatCoins(Vector3 world, int coins, float delay)`	`var t = HzUI.Label("Coins", Overlay, "+" + coins, 54, HzUI.Hex("#FFD54A"));`
`ShadowHandsGame.UI.cs:470 / void StepHintFor(int i)`	`if (_sim.StepCount > 1) Toast($"Step {i + 1} of {_sim.StepCount}", HzUI.Hex("#FFE27A"), 0.9f, 0.8f);`
`ShadowHandsGame.UI.cs:496 / void SkillUnlockPopup(int k, Action closed)`	`var p = ArtPopup("New jutsu!", $"{Skill.Name[k]}: {combo}\n{Skill.Desc[k]} {extra}", HzUI.Hex("#FF8A2A"), Art(SkillArt[k]),`
`ShadowHandsGame.UI.cs:497 / void SkillUnlockPopup(int k, Action closed)`	`("Got it", HzUI.Hex("#3FC795"), null));`
`ShadowHandsGame.UI.cs:506 / void BoosterUnlockPopup(int b, Action closed)`	`("Try it", HzUI.Hex("#3FC795"), null));`
`ShadowHandsGame.UI.cs:513 / void ThingPopup(string key, Action closed)`	`string title, msg; Sprite art = null; Color accent = HzUI.Hex("#7B61FF");`
`ShadowHandsGame.UI.cs:518 / void ThingPopup(string key, Action closed)`	`case "e3": title = "A Brute!"; msg = "Big and tough: watch his health bar.\nKeep the hits coming!"; accent = HzUI.Hex("#FF5A4E"); break;`
`ShadowHandsGame.UI.cs:519 / void ThingPopup(string key, Action closed)`	`case "p0": title = "Coin crates"; msg = "Shoot the gold-marked crates\nfor +15 coins."; art = null; accent = HzUI.Hex("#FFA51F"); break;`

`ShadowHandsGame.UI.cs:520 / void ThingPopup(string key, Action closed)`	`case "p1": title = "Villagers"; msg = "Villagers wear a green halo.\nNever hit them, aim around them!"; accent = HzUI.Hex("#3FC795"); break;`
`ShadowHandsGame.UI.cs:521 / void ThingPopup(string key, Action closed)`	`default: title = "Powder kegs"; msg = "Hit a red keg to blast every bandit near it.\nKeep villagers away from the blast!"; accent = HzUI.Hex("#FF5A4E"); break;`
`ShadowHandsGame.UI.cs:524 / void ThingPopup(string key, Action closed)`	`var p = ArtPopup(title, msg, accent, art, ("OK", HzUI.Hex("#3FC795"), null));`
`ShadowHandsGame.UI.cs:530 / void WinPopup(int earned)`	`ArtPopup("Level cleared!", $"+{earned} coins" + (_sim.BestChain >= 2 ? $" · best chain x{_sim.BestChain}" : ""), HzUI.Hex("#3FC795"), Art("win_badge"),`
`ShadowHandsGame.UI.cs:531 / void WinPopup(int earned)`	`("Next", HzUI.Hex("#2F6BFF"), NextLevel),`
`ShadowHandsGame.UI.cs:532 / void WinPopup(int earned)`	`("x2 coins", HzUI.Hex("#8A4DFF"), () =>`
`ShadowHandsGame.UI.cs:536 / void WinPopup(int earned)`	`if (ok) { Coins += earned; HzAudio.Play(Sfx.Coin, 0.8f); Toast($"+{earned} bonus coins!", HzUI.Hex("#FFE27A"), 1.2f, 0.62f); }`
`ShadowHandsGame.UI.cs:544 / void LosePopup(string reason)`	`ArtPopup("Defeated", reason, HzUI.Hex("#FF5A4E"), Art("lose_badge"),`
`ShadowHandsGame.UI.cs:545 / void LosePopup(string reason)`	`("Retry", HzUI.Hex("#FF5A4E"), Retry),`
`ShadowHandsGame.UI.cs:546 / void LosePopup(string reason)`	`("Skip level", HzUI.Hex("#8A4DFF"), () =>`
`ShadowHandsGame.UI.cs:550 / void LosePopup(string reason)`	`("Home", HzUI.Hex("#7B8794"), GoHome));`
`ShadowHandsGame.UI.cs:558 / void ShowSettingsMenu()`	`(HzAudio.Muted ? "Sound: OFF" : "Sound: ON", HzUI.Hex("#4C9EEB"), null),`
`ShadowHandsGame.UI.cs:559 / void ShowSettingsMenu()`	`(HzAudio.MusicMuted ? "Music: OFF" : "Music: ON", HzUI.Hex("#4C9EEB"), null),`
`ShadowHandsGame.UI.cs:560 / void ShowSettingsMenu()`	`("How to play", HzUI.Hex("#3FC795"), () => HzTween.Delay(0.15f, ShowHowTo)),`

`ShadowHandsGame.UI.cs:562 / void ShowSettingsMenu()`	`if (playing) { list.Add(("Restart", HzUI.Hex("#F5A623"), Retry)); list.Add(("Home", HzUI.Hex("#7B61FF"), GoHome)); }`
`ShadowHandsGame.UI.cs:563 / void ShowSettingsMenu()`	`list.Add(("Close", HzUI.Hex("#7B8794"), null));`
`ShadowHandsGame.UI.cs:564 / void ShowSettingsMenu()`	`var p = ArtPopup("Settings", "", HzUI.Hex("#4C9EEB"), null, list.ToArray());`
`ShadowHandsGame.UI.cs:577 / void ShowHowTo()`	`HzUI.Hex("#3FC795"), Art("sign_q"), ("Got it", HzUI.Hex("#3FC795"), null));`
`ShadowHandsGame.UI.cs:603 / void ShowHowTo()`	`var coin = HzUI.Panel("Coin", coins.transform, HzUI.Hex("#FFC531"), 30); coin.rectTransform.Place(new Vector2(1, 0.5f), new Vector2(60, 60), new Vector2(-42, 0));`
`ShadowHandsGame.UI.cs:611 / void ShowSkills()`	`var panel = ShopPanel("Jutsu Skills", "#6A35E8", "#3A1E9E");`
`ShadowHandsGame.UI.cs:639 / void ShowSkills()`	`var pip = HzUI.Panel("Pip", card.transform, i < lv && unlocked ? HzUI.Hex("#FFE14D") : new Color(1, 1, 1, 0.2f), 14);`
`ShadowHandsGame.UI.cs:645 / void ShowSkills()`	`var b = Pill(card.transform, label, lv >= 3 ? "#7B8794" : Coins >= Skill.UpgradeCost[lv] ? "#35C76B" : "#B0607A", () =>`
`ShadowHandsGame.UI.cs:656 / void ShowGloves()`	`var panel = ShopPanel("Gloves", "#2B6BD9", "#1B3A8C");`
`ShadowHandsGame.UI.cs:675 / void ShowGloves()`	`Pill(card.transform, label, eq ? "#7B8794" : own ? "#2F6BFF" : Coins >= GloveCost[i] ? "#35C76B" : "#B0607A", () =>`
`ShadowHandsGame.View.cs:13 / class/static declarations`	`new Theme { SkyTop = "#4FC3F0", Horizon = "#C9F3FF", Sea = "#27BFD3", Deck = "#C9C3B6", Trim = "#A3ACB4", Sun = "#FFF2DC", Accent = "#9AA4AE", Ground = "#5D8FA0" },`
`ShadowHandsGame.View.cs:14 / class/static declarations`	`new Theme { SkyTop = "#69CDE6", Horizon = "#E2F8DF", Sea = "#25B5A2", Deck = "#CDBE9C", Trim = "#9A8460", Sun = "#FFF6DE", Accent = "#6DBE45", Ground = "#4F8F6A" },`
`ShadowHandsGame.View.cs:15 / class/static declarations`	`new Theme { SkyTop = "#86AEE6", Horizon = "#F1F7FF", Sea = "#4CA3D8", Deck = "#D9E1EA", Trim = "#A6B4C4", Sun = "#F4F8FF", Accent = "#DDE8F3", Ground = "#7C93AE" },`

`ShadowHandsGame.View.cs:16 / class/static declarations`	`new Theme { SkyTop = "#FF8C6A", Horizon = "#FFDDA6", Sea = "#38A7C7", Deck = "#D7C1A3", Trim = "#B5805A", Sun = "#FFD9A8", Accent = "#E0443A", Ground = "#8A6A6A" },`
`ShadowHandsGame.View.cs:47 / void LoadMaterials()`	`_iceMat = Lit("#9FE6FF", 0.9f);`
`ShadowHandsGame.View.cs:116 / void SetupWorld()`	`_seaMat = Lit("#27BFD3", 0.85f);`
`ShadowHandsGame.View.cs:234 / void BuildDeck(int i, ShRng rng)`	`var wood = Lit("#C9873F", 0.25f); var woodDark = Lit("#8E5A2B", 0.2f);`
`ShadowHandsGame.View.cs:253 / void BuildDeck(int i, ShRng rng)`	`var post = Lit("#3B2F2A", 0.3f);`
`ShadowHandsGame.View.cs:254 / void BuildDeck(int i, ShRng rng)`	`var paper = Glow(_th.Accent == "#E0443A" ? "#FF7A3D" : "#FF5A4E", 1.3f);`
`ShadowHandsGame.View.cs:273 / void Dress(Transform deck, Vector3 at, ShRng rng)`	`Obj("Bamboo", deck, ShMeshes.Cylinder(8), Lit(k % 2 == 0 ? "#6DBE45" : "#58A83A", 0.4f), p, new Vector3(0.22f, hgt, 0.22f));`
`ShadowHandsGame.View.cs:274 / void Dress(Transform deck, Vector3 at, ShRng rng)`	`Obj("Leaf", deck, ShMeshes.Sphere(8), Lit("#7FD35A", 0.2f), p + new Vector3(0, hgt, 0), new Vector3(0.9f, 0.35f, 0.9f));`
`ShadowHandsGame.View.cs:278 / void Dress(Transform deck, Vector3 at, ShRng rng)`	`Obj("Trunk", deck, ShMeshes.Cylinder(8), Lit("#7A5A3C", 0.2f), at, new Vector3(0.35f, 1.0f, 0.35f));`
`ShadowHandsGame.View.cs:280 / void Dress(Transform deck, Vector3 at, ShRng rng)`	`Obj("Pine", deck, ShMeshes.Cylinder(10, 0.05f), Lit(k == 2 ? "#F4F8FF" : "#3E7D62", 0.2f), at + new Vector3(0, 0.8f + k * 0.9f, 0), new Vector3(2.2f - k * 0.6f, 1.4f, 2.2f - k * 0.6f));`
`ShadowHandsGame.View.cs:283 / void Dress(Transform deck, Vector3 at, ShRng rng)`	`var stone = Lit("#B9AFA3", 0.2f);`
`ShadowHandsGame.View.cs:286 / void Dress(Transform deck, Vector3 at, ShRng rng)`	`Obj("Light", deck, ShMeshes.RBox(0.7f, 0.6f, 0.7f, 0.08f), Glow("#FFB347", 1.2f), at + new Vector3(0, 1.9f, 0), Vector3.one, null, false);`
`ShadowHandsGame.View.cs:290 / void Dress(Transform deck, Vector3 at, ShRng rng)`	`var wood = Lit("#D08E45", 0.25f); var gold = Lit("#F5C33B", 0.6f, 0.3f);`
`ShadowHandsGame.View.cs:310 / void BuildBridge(int i)`	`var wood = Lit("#C9873F", 0.25f); var dark = Lit("#8E5A2B", 0.2f);`

`ShadowHandsGame.View.cs:313 / void BuildBridge(int i)`	`Obj("Plank", root, ShMeshes.RBox(3.2f, 0.16f, 0.55f, 0.04f), k % 2 == 0 ? wood : Lit("#BF7D38", 0.25f), new Vector3(0, -0.12f, (k + 0.5f) * len / planks), Vector3.one);`
`ShadowHandsGame.View.cs:324 / void BuildBackdrop(ShRng rng)`	`var rock = Lit(_th.Accent, 0.15f); var grey = Lit("#9FA7AE", 0.15f); var dark = Lit("#7D868F", 0.15f);`
`ShadowHandsGame.View.cs:336 / void BuildBackdrop(ShRng rng)`	`Obj("Hill", t, ShMeshes.Sphere(16), Lit("#5DAE4F", 0.1f), new Vector3(0, 0, 0), new Vector3(w * 2f, h * 1.2f, w * 2f), null, false);`
`ShadowHandsGame.View.cs:340 / void BuildBackdrop(ShRng rng)`	`Obj("Snow", t, ShMeshes.Cylinder(7, 0.02f), Lit("#F7FBFF", 0.3f), new Vector3(0, h * 1.6f * 0.62f, 0), new Vector3(w * 2.2f * 0.4f, h * 1.6f * 0.4f, w * 2.2f * 0.4f), null, false);`
`ShadowHandsGame.View.cs:345 / void BuildBackdrop(ShRng rng)`	`var red = Lit("#E0443A", 0.3f);`
`ShadowHandsGame.View.cs:348 / void BuildBackdrop(ShRng rng)`	`Obj("Top", t, ShMeshes.RBox(9.5f, 0.9f, 1.2f, 0.2f), Lit("#3B2F2A", 0.3f), new Vector3(0, 10.2f, 0), Vector3.one, null, false);`
`ShadowHandsGame.View.cs:353 / void BuildBackdrop(ShRng rng)`	`Obj("Block", t, ShMeshes.RBox(w, h, w * 0.8f, 0.4f), Lit("#C98E73", 0.15f), new Vector3(0, h * 0.5f, 0), Vector3.one, null, false);`
`ShadowHandsGame.View.cs:354 / void BuildBackdrop(ShRng rng)`	`Obj("Roof", t, ShMeshes.Cylinder(4, 0.15f), Lit("#5B3A3A", 0.2f), new Vector3(0, h, 0), new Vector3(w * 1.5f, 3f, w * 1.5f), Quaternion.Euler(0, 45, 0), false);`
`ShadowHandsGame.View.cs:361 / void BuildBackdrop(ShRng rng)`	`Obj("Slot", t, ShMeshes.RBox(w * 0.12f, h * 0.35f, 0.3f, 0.05f), Lit("#6F7880", 0.1f), new Vector3((s - 1) * w * 0.2f, h * 0.62f, -w * 0.4f), Vector3.one, null, false);`
`ShadowHandsGame.View.cs:436 / void BuildStepActors(int step)`	`static readonly string[] BodyHex = { "#7B3FA0", "#FF8A2A", "#6E7B8C", "#9E2B38" };`
`ShadowHandsGame.View.cs:437 / void BuildStepActors(int step)`	`static readonly string[] BandHexE = { "#E8484A", "#FFD23F", "#3D7BFF", "#1F1A24" };`
`ShadowHandsGame.View.cs:447 / void BuildStepActors(int step)`	`var skin = Lit("#F2C9A0", 0.3f); var pants = Lit("#3A2E4A", 0.3f); var black = Lit("#1E1A22", 0.5f);`

`ShadowHandsGame.View.cs:458 / void BuildStepActors(int step)`	`Obj("Mask", v.Head, ShMeshes.RBox(0.8f, 0.3f, 0.2f, 0.06f), Lit("#E0443A", 0.4f), new Vector3(0, -0.18f, 0.4f), Vector3.one);`
`ShadowHandsGame.View.cs:459 / void BuildStepActors(int step)`	`Obj("Horn", v.Head, ShMeshes.Cylinder(8, 0.05f), Lit("#F2E6C8", 0.4f), new Vector3(-0.3f, 0.4f, 0.1f), new Vector3(0.12f, 0.3f, 0.12f), Quaternion.Euler(0, 0, 25));`
`ShadowHandsGame.View.cs:460 / void BuildStepActors(int step)`	`Obj("Horn", v.Head, ShMeshes.Cylinder(8, 0.05f), Lit("#F2E6C8", 0.4f), new Vector3(0.3f, 0.4f, 0.1f), new Vector3(0.12f, 0.3f, 0.12f), Quaternion.Euler(0, 0, -25));`
`ShadowHandsGame.View.cs:471 / void BuildStepActors(int step)`	`Obj("Blade", v.ArmR, ShMeshes.RBox(0.05f, 0.55f, 0.1f, 0.02f), Lit("#DCE3EA", 0.8f, 0.6f), new Vector3(0, -0.45f, 0.25f), Vector3.one, Quaternion.Euler(70, 0, 0));`
`ShadowHandsGame.View.cs:473 / void BuildStepActors(int step)`	`Obj("Club", v.ArmR, ShMeshes.Cylinder(10, 1.5f), Lit("#7A4E2A", 0.3f), new Vector3(0, -0.45f, 0.1f), new Vector3(0.14f, 0.7f, 0.14f), Quaternion.Euler(80, 0, 0));`
`ShadowHandsGame.View.cs:476 / void BuildStepActors(int step)`	`var steel = Lit("#C3CCD6", 0.85f, 0.7f);`
`ShadowHandsGame.View.cs:485 / void BuildStepActors(int step)`	`v.BarFill = Obj("Fill", v.Bar, ShMeshes.Quad(), UnlitMat(HzUI.Hex("#FF4D5A")), new Vector3(0, 0, -0.01f), new Vector3(2.3f, 0.19f, 1), null, false);`
`ShadowHandsGame.View.cs:515 / void BuildStepActors(int step)`	`var wood = Lit("#D08E45", 0.25f); var dark = Lit("#8E5A2B", 0.25f);`
`ShadowHandsGame.View.cs:519 / void BuildStepActors(int step)`	`Obj("Coin", root, ShMeshes.Cylinder(18), Lit("#FFC531", 0.8f, 0.5f), new Vector3(0, 0.45f, 0.46f), new Vector3(0.45f, 0.06f, 0.45f), Quaternion.Euler(90, 0, 0));`
`ShadowHandsGame.View.cs:520 / void BuildStepActors(int step)`	`Obj("CoinDot", root, ShMeshes.RBox(0.12f, 0.22f, 0.04f, 0.02f), Lit("#E09A12", 0.6f), new Vector3(0, 0.45f, 0.5f), Vector3.one);`
`ShadowHandsGame.View.cs:523 / void BuildStepActors(int step)`	`var red = Lit("#C93A2E", 0.45f); var band = Lit("#2B2B2B", 0.6f, 0.4f);`

ShadowHandsGame.View.cs:527 / void BuildStepActors(int step)	Obj("Mark", root, ShMeshes.RBox(0.32f, 0.32f, 0.04f, 0.04f), Lit("#FFD23F", 0.4f), new Vector3(0, 0.5f, 0.37f), Vector3.one, Quaternion.Euler(0, 0, 45));
ShadowHandsGame.View.cs:528 / void BuildStepActors(int step)	Obj("Fuse", root, ShMeshes.Cylinder(6), Lit("#E8D9B0", 0.3f), new Vector3(0, 1f, 0), new Vector3(0.05f, 0.25f, 0.05f));
ShadowHandsGame.View.cs:529 / void BuildStepActors(int step)	Obj("Spark", root, ShMeshes.Sphere(8), Glow("#FFB347", 2.2f), new Vector3(0, 1.27f, 0), Vector3.one * 0.12f, null, false);
ShadowHandsGame.View.cs:532 / void BuildStepActors(int step)	var cloth = Lit("#8EC9E8", 0.3f); var skin = Lit("#F2C9A0", 0.3f); var straw = Lit("#E8C66A", 0.3f);
ShadowHandsGame.View.cs:536 / void BuildStepActors(int step)	Obj("EyeL", root, ShMeshes.Sphere(8), Lit("#1E1A22", 0.5f), new Vector3(-0.1f, 1.18f, 0.23f), new Vector3(0.07f, 0.1f, 0.05f), null, false);
ShadowHandsGame.View.cs:537 / void BuildStepActors(int step)	Obj("EyeR", root, ShMeshes.Sphere(8), Lit("#1E1A22", 0.5f), new Vector3(0.1f, 1.18f, 0.23f), new Vector3(0.07f, 0.1f, 0.05f), null, false);
ShadowHandsGame.View.cs:540 / void BuildStepActors(int step)	Limb(root, "LegL", new Vector3(-0.1f, 0.45f, 0), 0.13f, 0.4f, Lit("#5A6B7A", 0.3f));
ShadowHandsGame.View.cs:541 / void BuildStepActors(int step)	Limb(root, "LegR", new Vector3(0.1f, 0.45f, 0), 0.13f, 0.4f, Lit("#5A6B7A", 0.3f));
ShadowHandsGame.View.cs:543 / void BuildStepActors(int step)	Obj("Halo", root, ShMeshes.Cylinder(20), Glow("#7CFF9E", 1.4f), new Vector3(0, 1.85f, 0), new Vector3(0.5f, 0.03f, 0.5f), null, false);
ShadowHandsGame.View.cs:644 / void ExplodeEnemy(int id)	Burst(c + Vector3.up * 0.4f * v.Scale, "#F2C9A0", boss ? 8 : 4, 0.14f * v.Scale, 4f, true);
ShadowHandsGame.View.cs:645 / void ExplodeEnemy(int id)	Puff(c, "#FFFFFF", 1.4f * v.Scale, 0.35f);
ShadowHandsGame.View.cs:646 / void ExplodeEnemy(int id)	if (boss) { Shake(0.5f); Puff(c, "#FFB347", 3.5f, 0.45f); }
ShadowHandsGame.View.cs:657 / void OnArmorBreakView(int id)	Burst(h.position, "#DCE3EA", 6, 0.08f, 4f, true);

`ShadowHandsGame.View.cs:686 / void OnHitView(int id, int kind, V2 pos)`	`case Skill.Fire: Puff(p, "#FF8A2A", 1.6f, 0.3f); Burst(p, "#FFB347", 8, 0.1f, 4f, false); HzAudio.Play(Sfx.FireHit, 0.65f); break;`
`ShadowHandsGame.View.cs:687 / void OnHitView(int id, int kind, V2 pos)`	`case Skill.Stars: Burst(p, "#E8EEF5", 6, 0.06f, 3.5f, false); HzAudio.Play(Sfx.StarHit, 0.6f); break;`
`ShadowHandsGame.View.cs:688 / void OnHitView(int id, int kind, V2 pos)`	`case Skill.Thunder: Puff(p, "#B07CFF", 1.8f, 0.3f); Burst(p, "#FFE45C", 8, 0.07f, 5f, false); HzAudio.Play(Sfx.ThunderHit, 0.7f); break;`
`ShadowHandsGame.View.cs:696 / void OnCrateView(int id, int coins, V2 pos)`	`Burst(p, "#D08E45", 10, 0.14f, 4f, true);`
`ShadowHandsGame.View.cs:697 / void OnCrateView(int id, int coins, V2 pos)`	`Burst(p + Vector3.up * 0.3f, "#FFC531", 6, 0.1f, 5f, true);`
`ShadowHandsGame.View.cs:705 / void OnKegView(int id, V2 pos)`	`Puff(p, "#FF8A2A", ShSim.KegRadius * 2.2f, 0.5f);`
`ShadowHandsGame.View.cs:706 / void OnKegView(int id, V2 pos)`	`Puff(p, "#FFE07A", ShSim.KegRadius * 1.2f, 0.3f);`
`ShadowHandsGame.View.cs:707 / void OnKegView(int id, V2 pos)`	`Burst(p, "#C93A2E", 10, 0.12f, 6f, true);`
`ShadowHandsGame.View.cs:708 / void OnKegView(int id, V2 pos)`	`Burst(p, "#3B3B3B", 10, 0.1f, 7f, true);`
`ShadowHandsGame.View.cs:715 / void OnVillagerHitView(int id)`	`Burst(t.position + Vector3.up, "#FF4D5A", 8, 0.08f, 3f, false);`
`ShadowHandsGame.View.cs:737 / void SpawnProjView(int id, int kind)`	`t = Obj("Star", _fxRoot, ShMeshes.Star(), Lit("#E2E8EE", 0.9f, 0.8f), Vector3.zero, Vector3.one * 0.5f, null, false);`
`ShadowHandsGame.View.cs:740 / void SpawnProjView(int id, int kind)`	`t = Obj("Thunder", _fxRoot, ShMeshes.Sphere(12), Glow("#B07CFF", 2.6f), Vector3.zero, Vector3.one * 0.42f, null, false);`
`ShadowHandsGame.View.cs:741 / void SpawnProjView(int id, int kind)`	`Obj("Core", t, ShMeshes.Sphere(10), Glow("#FFFFFF", 2.0f), Vector3.zero, Vector3.one * 0.5f, null, false);`
`ShadowHandsGame.View.cs:744 / void SpawnProjView(int id, int kind)`	`t = Obj("Fire", _fxRoot, ShMeshes.Sphere(12), Glow("#FF7A1F", 2.8f), Vector3.zero, Vector3.one * 0.45f, null, false);`

`ShadowHandsGame.View.cs:745 / void SpawnProjView(int id, int kind)`	`Obj("Core", t, ShMeshes.Sphere(10), Glow("#FFE07A", 2.2f), Vector3.zero, Vector3.one * 0.55f, null, false);`
`ShadowHandsGame.View.cs:758 / void EndProjView(int id, int how, V2 pos)`	`if (how == 0) Puff(W(_viewStep, pos, 1.2f), pv.kind == Skill.Fire ? "#FF8A2A" : "#B07CFF", 0.8f, 0.2f);`
`ShadowHandsGame.View.cs:775 / void UpdateProjViews(float dt)`	`if (trail && pv.kind != Skill.Stars) Spark(pv.t.position, pv.kind == Skill.Fire ? "#FF9A3D" : "#C9A2FF", 0.28f, 0.25f);`
`ShadowHandsGame.View.cs:786 / void OnBladeView(float range)`	`var arc = Obj("Arc", root, ShMeshes.Arc(0.7f, 1f, ShSim.BladeHalfAngle), Glow("#BFF6FF", 2.2f), Vector3.zero, Vector3.one * range, null, false);`
`ShadowHandsGame.View.cs:805 / void BoltSeg(Vector3 a, Vector3 b)`	`var t = Obj("Bolt", _fxRoot, ShMeshes.RBox(1, 1, 1, 0.2f), Glow("#D9B8FF", 2.6f), (a + b) * 0.5f, new Vector3(0.08f, 0.08f, d.magnitude), Quaternion.LookRotation(d.normalized), false);`
`ShadowHandsGame.View.cs:814 / void BuildWolf()`	`var g = Glow("#3FF2D2", 1.6f); var w = Glow("#E6FFFB", 1.4f);`
`ShadowHandsGame.View.cs:834 / void OnWolfView(int id, V2 pos)`	`Puff(_wolf.position + Vector3.up * 0.6f, "#3FF2D2", 2f, 0.35f);`
`ShadowHandsGame.View.cs:841 / void OnWolfView(int id, V2 pos)`	`Puff(_wolfTo + Vector3.up * 0.8f, "#3FF2D2", 1.6f, 0.3f);`
`ShadowHandsGame.View.cs:857 / void UpdateWolf(float dt)`	`if (Random.value < 0.4f) Spark(_wolf.position + Vector3.up * 0.7f + Random.insideUnitSphere * 0.4f, "#3FF2D2", 0.2f, 0.35f);`
`ShadowHandsGame.View.cs:861 / void UpdateWolf(float dt)`	`if (_wolfLife <= 0) { Puff(_wolf.position + Vector3.up * 0.6f, "#3FF2D2", 1.8f, 0.3f); _wolf.gameObject.SetActive(false); }`
`ShadowHandsGame.View.cs:871 / void OnFrostView()`	`Spark(p, "#DFF6FF", Random.Range(0.12f, 0.22f), Random.Range(1.2f, 2.2f), new Vector3(Random.Range(-0.3f, 0.3f), -1.2f, 0));`
`ShadowHandsGame.View.cs:884 / void OnMeteorView()`	`var rock = Obj("Meteor", _fxRoot, ShMeshes.Sphere(10), Glow("#FF6A1F", 2.4f), land + new Vector3(4, 16, -2), Vector3.one * 0.9f, null, false);`

`ShadowHandsGame.View.cs:886 / void OnMeteorView()`	`HzTween.Run(rock, "fall", 0.5f, k => { if (rock) rock.position = Vector3.Lerp(from, land, k); if (Random.value < 0.6f) Spark(rock.position, "#FFB347", 0.4f, 0.3f); },`
`ShadowHandsGame.View.cs:890 / void OnMeteorView()`	`Puff(land, "#FF8A2A", 3f, 0.45f);`
`ShadowHandsGame.View.cs:891 / void OnMeteorView()`	`Burst(land, "#5A4A44", 8, 0.14f, 5f, true);`

CHAPTER 10

Reskin recipe: visible menu and HUD text

Open `Scripts/ShadowHandsGame.UI.cs`, find the displayed literal `"Settings"`, and replace its displayed value with `"Options"`. Keep hierarchy names and technical IDs unchanged. In HzUI.Label/Button calls the first string may be an object name; change the displayed text argument. Read the method references below to distinguish construction from later refresh assignments.

Preserve interpolation braces, invariant number formatting, ON/OFF updates and line breaks. Update both initial and refresh paths, portrait and landscape. Exact helper names/unlocks/charges: Scripts/ShData.cs; help and effect descriptions: Scripts/ShData.cs. Level names/hints remain separate JSON content. If a title appears in art, Title property or another screen, update those visible uses deliberately without renaming GameId, namespace, resources or save keys.

Hard-coded game strings do not automatically use shared localization. To translate the interface, route every display assignment through your translation table. Dictionaries, word banks and theme pools are independent gameplay assets and need separate validation. Test help, settings, results, tutorial and helper introductions after a fresh launch.

Source	Exact current code
`ShadowHandsGame.cs:242 / public void PressSign(int k)`	`if (k == Skill.SignR && !_sim.SignROn) { HzAudio.Play(Sfx.Wrong, 0.5f); Toast("R unlocks at level " + Skill.RUnlock, null, 1.2f, 0.62f); return; }`
`ShadowHandsGame.cs:317 / public void UseBooster(int b)`	`if (Level < Booster.Unlock[b]) { HzAudio.Play(Sfx.Wrong, 0.5f); Toast($"{Booster.Name[b]} unlocks at level {Booster.Unlock[b]}", null, 1.3f, 0.62f); return; }`
`ShadowHandsGame.cs:326 / public void UseBooster(int b)`	`if (ok) { SetCharges(b, Charges(b) + 1); HzAudio.Play(Sfx.Coin, 0.7f); Toast($"+1 {Booster.Name[b]}", HzUI.Hex("#FFE27A"), 1.2f, 0.62f); RefreshHud(); }`
`ShadowHandsGame.cs:369 / IEnumerator LoseFlow()`	`LosePopup(_sim.LoseReason == "villager" ? "You hit a villager! Aim around them." : "A bandit reached you! Cast faster, and look around.");`
`ShadowHandsGame.cs:390 / public bool TryUpgrade(int k)`	`public static readonly string[] GloveName = { "Shadow", "Crimson", "Jade", "Gold", "Violet" };`

`ShadowHandsGame.cs:417 / public override string DebugCommand(string cmd)`	`$"won={_sim.Won} lost={_sim.Lost} paused={_paused} tut={_tut} guide={_guide} coins={Coins} runCoins={_sim.Coins} hold={_sim.Hold} goal=[{(_goal ? _goalText.text.Replace('\n', ' ') : "")}] hand={(_hand && _hand.gameObject.activeSelf)}";`
`ShadowHandsGame.UI.cs:46 / class/static declarations`	`var b = HzUI.Button("Pill", parent, text, HzUI.Hex(hex), Color.white, onClick, font, (int)(size.y * 0.5f));`
`ShadowHandsGame.UI.cs:61 / void CoinPill(Transform parent, Vector2 anchor, Vector2 pos)`	`_coinText = HzUI.Label("N", bg.transform, Coins.ToString(), 54, Color.white, TextAnchor.MiddleRight);`
`ShadowHandsGame.UI.cs:94 / void BuildHome()`	`var t = HzUI.Label("Title", _home, "JUTSU DEFENDER", 120, Color.white);`
`ShadowHandsGame.UI.cs:97 / void BuildHome()`	`var lv = HzUI.Label("Lv", _home, $"Level {Level} · {_def.name}", p ? 56 : 50, Color.white);`
`ShadowHandsGame.UI.cs:109 / void BuildHome()`	`var t = HzUI.Label("!", dot.transform, "!", 36, Ink); t.rectTransform.Stretch();`
`ShadowHandsGame.UI.cs:112 / void BuildHome()`	`var help = HzUI.Label("Help", _home, p ? "Tap the hand signs · drag to look" : "Q W E R: hand signs · drag or A / D: look around", p ? 40 : 36, new Color(1, 1, 1, 0.9f));`
`ShadowHandsGame.UI.cs:123 / void BuildHome()`	`var t = HzUI.Label("L", b.transform, label, 52, Color.white); t.rectTransform.Place(new Vector2(0.5f, 0.5f), new Vector2(230, 100), new Vector2(-26, 0));`
`ShadowHandsGame.UI.cs:149 / void BuildHud()`	`_levelText = HzUI.Label("Level", _hud, $"Level {Level}", 60, Color.white);`
`ShadowHandsGame.UI.cs:193 / void BuildComboScroll(bool portrait)`	`var l = HzUI.Label("Lock", row.transform, $"Lv {Skill.Unlock[k]}", 38, new Color(1, 1, 1, 0.75f));`
`ShadowHandsGame.UI.cs:207 / void BuildComboScroll(bool portrait)`	`var t = HzUI.Label("K", chip.transform, Skill.SignKey[sign], 34, Color.white); t.rectTransform.Stretch();`
`ShadowHandsGame.UI.cs:211 / void BuildComboScroll(bool portrait)`	`var name = HzUI.Label("Name", row.transform, Skill.Name[k], 24, new Color(1, 1, 1, 0.85f), TextAnchor.MiddleCenter, false);`

`ShadowHandsGame.UI.cs:241 / void BuildSigns(bool portrait)`	`var b = HzUI.Button("Sign" + k, _hud, null, locked ? new Color(0.45f, 0.45f, 0.5f) : HzUI.Hex(SignHex[k]), Color.white, () => PressSign(kk), 40, (int)(size * 0.5f));`
`ShadowHandsGame.UI.cs:258 / void BuildSigns(bool portrait)`	`var kt = HzUI.Label("K", badge.transform, Skill.SignKey[k], 44, Ink); kt.rectTransform.Stretch();`
`ShadowHandsGame.UI.cs:263 / void BuildSigns(bool portrait)`	`var lv = HzUI.Label("Lv", b.transform, $"Lv {Skill.RUnlock}", 40, Color.white); lv.rectTransform.Place(new Vector2(0.5f, 0), new Vector2(160, 50), new Vector2(0, 36));`
`ShadowHandsGame.UI.cs:279 / void BuildBoosters(bool portrait)`	`var btn = HzUI.Button("Boost" + b, _hud, null, HzUI.Hex(BoostHex[b]), Color.white, () => UseBooster(bb), 40, (int)(size * 0.32f));`
`ShadowHandsGame.UI.cs:286 / void BuildBoosters(bool portrait)`	`var bt = HzUI.Label("N", badge.transform, "0", 36, Color.white); bt.rectTransform.Stretch();`
`ShadowHandsGame.UI.cs:290 / void BuildBoosters(bool portrait)`	`var lv = HzUI.Label("Lv", lockBg.transform, $"Lv {Booster.Unlock[b]}", (int)(size * 0.24f), Color.white); lv.rectTransform.Place(new Vector2(0.5f, 0), new Vector2(size, size * 0.34f), new Vector2(0, size * 0.18f));`
`ShadowHandsGame.UI.cs:293 / void BuildBoosters(bool portrait)`	`var name = HzUI.Label("Name", btn.transform, Booster.Name[b], 26, Color.white, TextAnchor.MiddleRight, false);`
`ShadowHandsGame.UI.cs:313 / void RefreshHud()`	`t.text = c > 0 ? c.ToString() : "+";`
`ShadowHandsGame.UI.cs:379 / void LevelBanner()`	`var t = HzUI.Label("T", bg.transform, $"Level {Level}", 86, Color.white); t.rectTransform.Place(new Vector2(0.5f, 1), new Vector2(800, 100), new Vector2(0, -60));`
`ShadowHandsGame.UI.cs:380 / void LevelBanner()`	`var s = HzUI.Label("S", bg.transform, _def.name, 46, HzUI.Hex("#FFE27A")); s.rectTransform.Place(new Vector2(0.5f, 0), new Vector2(800, 70), new Vector2(0, 40));`
`ShadowHandsGame.UI.cs:394 / void Banner(string text, Color color)`	`var t = HzUI.Label("T", band.transform, text, 150, color); t.rectTransform.Stretch();`
`ShadowHandsGame.UI.cs:416 / void ShowGoal(string text)`	`_goalText = HzUI.Label("T", bg.transform, "", IsPortrait ? 50 : 46, Color.white);`

ShadowHandsGame.UI.cs:429 / void ChainPop(int n)	var t = HzUI.Label("Chain", Overlay, $"CHAIN x{n}", 90, HzUI.Hex("#FFE14D"));
ShadowHandsGame.UI.cs:445 / void FloatCoins(Vector3 world, int coins, float delay)	var t = HzUI.Label("Coins", Overlay, "+" + coins, 54, HzUI.Hex("#FFD54A"));
ShadowHandsGame.UI.cs:470 / void StepHintFor(int i)	if (_sim.StepCount > 1) Toast($"Step {i + 1} of {_sim.StepCount}", HzUI.Hex("#FFE27A"), 0.9f, 0.8f);
ShadowHandsGame.UI.cs:496 / void SkillUnlockPopup(int k, Action closed)	var p = ArtPopup("New jutsu!", $"{Skill.Name[k]}: {combo}\n{Skill.Desc[k]}{extra}", HzUI.Hex("#FF8A2A"), Art(SkillArt[k]),
ShadowHandsGame.UI.cs:505 / void BoosterUnlockPopup(int b, Action closed)	var p = ArtPopup("Booster unlocked!", $"{Booster.Name[b]}\n{Booster.Desc[b]}\n{Booster.Free[b]} free uses!", HzUI.Hex(BoostHex[b]), Art(BoostArt[b]),
ShadowHandsGame.UI.cs:524 / void ThingPopup(string key, Action closed)	var p = ArtPopup(title, msg, accent, art, ("OK", HzUI.Hex("#3FC795"), null));
ShadowHandsGame.UI.cs:530 / void WinPopup(int earned)	ArtPopup("Level cleared!", $"+{earned} coins" + (_sim.BestChain >= 2 ? $" · best chain x{_sim.BestChain}" : ""), HzUI.Hex("#3FC795"), Art("win_badge"),
ShadowHandsGame.UI.cs:536 / void WinPopup(int earned)	if (ok) { Coins += earned; HzAudio.Play(Sfx.Coin, 0.8f); Toast($"+{earned} bonus coins!", HzUI.Hex("#FFE27A"), 1.2f, 0.62f); }
ShadowHandsGame.UI.cs:544 / void LosePopup(string reason)	ArtPopup("Defeated", reason, HzUI.Hex("#FF5A4E"), Art("lose_badge"),
ShadowHandsGame.UI.cs:558 / void ShowSettingsMenu()	(HzAudio.Muted ? "Sound: OFF" : "Sound: ON", HzUI.Hex("#4C9EEB"), null),
ShadowHandsGame.UI.cs:559 / void ShowSettingsMenu()	(HzAudio.MusicMuted ? "Music: OFF" : "Music: ON", HzUI.Hex("#4C9EEB"), null),
ShadowHandsGame.UI.cs:564 / void ShowSettingsMenu()	var p = ArtPopup("Settings", "", HzUI.Hex("#4C9EEB"), null, list.ToArray());
ShadowHandsGame.UI.cs:567 / void ShowSettingsMenu()	sound.onClick.AddListener(() => { HzAudio.Muted = !HzAudio.Muted; sound.SetText(HzAudio.Muted ? "Sound: OFF" : "Sound: ON"); HzAudio.Play(Sfx.Toggle, 0.7f); });

ShadowHandsGame.UI.cs:570 / void ShowSettingsMenu()	music.onClick.AddListener(() => { HzAudio.MusicMuted = !HzAudio.MusicMuted; music.SetText(HzAudio.MusicMuted ? "Music: OFF" : "Music: ON"); HzAudio.Play(Sfx.Toggle, 0.7f); });
ShadowHandsGame.UI.cs:575 / void ShowHowTo()	ArtPopup("How to play",
ShadowHandsGame.UI.cs:597 / void ShowHowTo()	var back = HzIcons.IconButton(_panel, HzIcons.ArrowBack, new Color(1, 1, 1, 0.22f), () => { HzAudio.Play("popup_close", 0.6f); Destroy(_panel.gameObject); _panel = null; if (_mode == Mode.Home) BuildHome(); }, 110);
ShadowHandsGame.UI.cs:599 / void ShowHowTo()	var t = HzUI.Label("Title", _panel, title, 84, Color.white); t.rectTransform.Place(new Vector2(0.5f, 1), new Vector2(900, 120), new Vector2(0, -85));
ShadowHandsGame.UI.cs:604 / void ShowHowTo()	var ct = HzUI.Label("N", coins.transform, Coins.ToString(), 54, Color.white, TextAnchor.MiddleRight); ct.rectTransform.Place(new Vector2(0.5f, 0.5f), new Vector2(170, 80), new Vector2(-40, 0));
ShadowHandsGame.UI.cs:624 / void ShowSkills()	var nm = HzUI.Label("Name", card.transform, Skill.Name[k], 56, Color.white, TextAnchor.MiddleLeft);
ShadowHandsGame.UI.cs:631 / void ShowSkills()	var ctt = HzUI.Label("K", chip.transform, Skill.SignKey[sign], 36, Color.white); ctt.rectTransform.Stretch();
ShadowHandsGame.UI.cs:633 / void ShowSkills()	var desc = HzUI.Label("Desc", card.transform, unlocked ? Skill.Desc[k] + "\n" + Skill.UpgradeDesc[k] : $"Unlocks at level {Skill.Unlock[k]}", p ? 30 : 25, new Color(1, 1, 1, 0.85f), TextAnchor.UpperLeft, false);
ShadowHandsGame.UI.cs:648 / void ShowSkills()	else if (SkillLv(kk) < 3) Toast("Not enough coins. Clear levels to earn more!", null, 1.3f, 0.5f);
ShadowHandsGame.UI.cs:673 / void ShowGloves()	var nm = HzUI.Label("Name", card.transform, GloveName[i], 50, Color.white); nm.rectTransform.Place(new Vector2(0.5f, 1), new Vector2(320, 70), new Vector2(0, -270));
ShadowHandsGame.UI.cs:678 / void ShowGloves()	else Toast("Not enough coins yet!", null, 1.2f, 0.5f);

`ShadowHandsGame.View.cs:118 / void SetupWorld()`	`_seaMat.SetTexture("_BaseMap", SeaTexture());`
`ShadowHandsGame.View.cs:119 / void SetupWorld()`	`_seaMat.SetTextureScale("_BaseMap", new Vector2(70, 70));`
`ShadowHandsGame.View.cs:172 / void ApplyTheme(int theme)`	`_skyMat.SetTexture("_BaseMap", Gradient(top, hor, Color.Lerp(hor, sea, 0.5f)));`
`ShadowHandsGame.View.cs:559 / void UpdateView(float dt)`	`_seaMat.SetTextureOffset("_BaseMap", new Vector2(Time.time * 0.012f, Time.time * 0.008f));`
`ShData.cs:74 / class/static declarations`	`public static readonly string[] Name = { "Bandit", "Dasher", "Ironhat", "Brute" };`
`ShData.cs:85 / public static int DefaultHp(int type) => type == Brute ? 8 : 1;`	`public static readonly string[] Name = { "Coin crate", "Villager", "Powder keg" };`
`ShData.cs:93 / public static int DefaultHp(int type) => type == Brute ? 8 : 1;`	`public static readonly string[] Name = { "Fire Orb", "Star Fan", "Blade Arc", "Thunder Orb" };`
`ShData.cs:120 / public static int DefaultHp(int type) => type == Brute ? 8 : 1;`	`public static readonly string[] Name = { "Spirit Wolf", "Frost Scroll", "Meteor Rain" };`
`ShData.cs:133 / public static int DefaultHp(int type) => type == Brute ? 8 : 1;`	`public static readonly string[] Name = { "Lagoon Pier", "Bamboo Isle", "Snow Shrine", "Sunset Temple" };`

CHAPTER 11

Artwork, fonts, audio and licences

Replace game images under Resources/shadowhands through Unity while retaining resource keys, .meta GUIDs, Sprite/UI import settings, alpha, pivots and slicing. Procedural graphics in the art/view sources may need separate colour changes. Replacing one PNG does not change every fallback. Keep third-party font and other licence notices with the corresponding shipped assets.

Effects are under Resources/shadowhands/Sfx. Preserve each actual load key and importer settings when replacing sounds. Supplied music: Resources/shadowhands/Music/dojo_loop.ogg. Preserve that load key and verify loop/compression and the independent Music toggle. Kit UI uses supplied fonts; preserve actual licence notices and verify new glyphs.

CHAPTER 12

Local saves, reset and migration

HzSave wraps PlayerPrefs and flushes local state on focus loss/quit. WebGL persistence uses browser storage tied to the hosting origin. Clearing site data or changing origin starts a new local save; this is not cloud synchronization. Inspect HzSave/PlayerPrefs calls in the actual game source before altering progression, stars, helper charges or tutorial flags.

Verify fresh launch, a completed arena, helper use, sound options, focus loss and restart. Use a dedicated test profile for resets. Keep technical save keys stable during colour/text reskins. If changing schema or keys, implement and test a migration rather than silently discarding existing users' progress.

CHAPTER 13

Advertising integration and configuration

Asset: `Assets/Szekipapa77Ads/Resources/AdsSettings.asset`. Inspect backend, useGoogleTestIds and missingSdkFallback in the publisher-owned Ads configuration. Buyer projects include adapter/integration code and exclude external provider SDK binaries. WebGL and a missing-provider fallback simulate ads; these are not live commercial impressions.

For real Android ads install the supported provider SDK from its official source into your working project, enter your own app/placement IDs and use test ads first. Verify banner/interstitial/rewarded callbacks on supported hardware. For production use your own IDs and disable simulated missing-SDK fallback. Build tooling rejects unsafe production fallback settings. No IAP, remote backend or HLK is included in this publisher edition.

CHAPTER 14

Android build and release

Install Unity 6000.0.62f1 Android modules through Hub. Select Android, run Setup and review application ID `com.szekipapa77.shadowhands`, version, scene orientation and signing. The build command writes `Builds/Android/shadowhands.apk`. Confirm BuildResult.Succeeded and the actual output file; intermediate report totals are not the final APK size.

Use your own keystore/signing and distribution settings. A test-signed demo is not a signed store release. A provider-free build simulates ads; a provider test-demo does not prove production ad-account configuration. Test real hardware touch, safe areas, focus/audio, save/relaunch and reward callbacks. Keep passwords, keystores and private service credentials out of buyer packages.

CHAPTER 15

WebGL build and hosting

Install matching WebGL support. Run Setup and Build WebGL; output is `Builds/WebGL/shadowhands/`. Serve index.html with all loader/framework/data/WASM/StreamingAssets files over HTTP(S), not file://. Match compression, Content-Encoding and MIME types to the host. Outer page sizing and actual scene orientation are different settings.

The publisher's playable demo is hosted on HyperCodeStore. Upload the actual runtime files rather than merely changing a URL. Test a fresh anonymous load, real pointer controls, audio after interaction, saved progress/reload and phone/desktop sizing. Rebuild and rehost when code/assets change. A successful HTTP page alone is not gameplay verification.

CHAPTER 16

Troubleshooting

Compile errors: restore matching packages and the full private Kit/Ads layer; only one game Kit belongs in a project. **Pink/invisible rendering:** run Setup and inspect Graphics and every Quality URP assignment. **No input:** classic input handling must be enabled; check canvas focus and blocking modals. **Missing resources:** restore exact paths, .meta GUIDs and importer settings.

Edits do not appear: stop Play, Save in the level editor, reload the canonical bank and check current saved level. **Text reverts:** update construction and refresh assignments from the exact table. **No live ads:** provider binaries are deliberately excluded; inspect your installed provider and fallback mode. **Scene reload exception:** retain owner-lifetime and generation guards on delayed callbacks. **WebGL fails:** inspect browser Console, compression headers and missing payloads.

CHAPTER 17

Support and final release checklist

Publisher: Szekipapa77. Support: szekipapa77@gmail.com. Include game title, exact Unity version, platform, first Console error and reproduction steps. Never send passwords/tokens.

1. Clean-import the standalone project and native package.
2. Play both scenes and replace a scene near startup.
3. Review tutorial, normal completion/retry and every helper activation plus effect.
4. Validate editor changes and test reskin text/colours in both layouts.
5. Rebuild affected Android/WebGL outputs.
6. Verify complete public APK bytes, actual browser demo, documentation and video links.
7. Keep licences and exclude all internal prompts, planning/capture/build/QA tooling from buyers.